12 Databáze MySQL

V této kapitole se dozvíte:

- Základní informace o databázích MySQL.
- Jak se připojit k databázovému serveru.
- Jak spouštět SQL příkazy na databázovém serveru.
- Jak uvolnit výsledky a uzavřít připojení databázi.

Po jejím prostudování byste měli být schopni:

- Umět používat rozhraní mysqli pro práci s databází MySQL.
- Umět spouštět libovolné příkazy SQL.
- Umět zpracovat výsledky, které vrátil příkaz SELECT.

Klíčová slova této kapitoly:

MySQL, mysqli, mysqli_real_connect, mysqli_select_db mysqli_query, mysqli_close mysqli_fetch_row, mysqli_fetch_array, mysqli_fetch_assoc, mysqli_fetch_object, mysqli_free_result.

Doba potřebná ke studiu: 4 hodiny



Průvodce studiem

Studium této kapitoly není náročné. V úvodu kapitoly se krátce seznámíte s databázovým serverem MySQL. Dále se již naučíte pracovat s tímto serverem pomocí jazyka PHP. Práce s databází spočívá ve čtyřech krocích – připojení k databázovému serveru, výběr databáze, spouštění příkazů a uzavření připojení k databázovému serveru.

Databázový server MySQL slouží k vytváření relačních databází. Je k dispozici v bezplatné verzi pro nekomerční a soukromé použití. Pro komerční použití existuje i placená varianta databázového serveru MySQL. Databáze se vždy skládá z tabulek, které jsou navzájem propojeny relacemi. V této opoře se nebudeme zabývat teoretickými východisky relačních databází. Zaměříme se pouze na tvorbu databází v MySQL a na práci s databází MySQL prostřednictvím jazyka PHP. Práce s databázovým serverem je možná pomocí různých nástrojů. Jedním z nich je phpMyAdmin, který je vytvořen v jazyce PHP a je k dispozici zdarma. Prostřednictvím internetového prohlížeče můžeme pomocí tohoto nástroje vytvářet databáze, tabulky, plnit je daty, dotazovat se apod.

12.1 Typy tabulek a datové typy

Specialitou MySQL je možnost zadání typu tabulky v okamžiku jejího vytvoření. Jednotlivé typy tabulek se liší různými vlastnostmi. Mezi nejdůležitější typy tabulek patří MyISAM, InnoDB a HEAP.

Tabulka typu MyISAM představuje v MySQL standardní typ tabulek. Jedná se o stabilní, vyspělý a jednoduše spravovatelný typ tabulek. Pokud není žádný speciální důvod pro použití jiného typu tabulky, je tento typ ideální.



Tabulka typu InnoDB má všechny vlastnosti typu MyISAM a navíc má dvě odlišnosti. První odlišností je, že databázové operace v tabulkách typu InnoDB je možné spouštět jako transakce. Druhou odlišností je, že tabulky InnoDB podporují používaní pravidel integrity. Kromě toho mají ale tabulky InnoDB i nevýhody. První nevýhodou je, že nelze vytvořit fulltextový index. Druhou je, že správa tabulek InnoDB je o něco složitější. Poslední nevýhodou je, že komerční licence MySQL s podporou InnoDB je dražší než verze bez ní.

Posledním typem jsou tabulky typu HEAP, které se vytváří pouze v operační paměti RAM. Neukládají se tedy na pevný disk. Používají tzv. index typu hash, který umožňuje rychlý přístup k záznamům.

Každý sloupec tabulky v relační databázi musí mít určený datový typ, který je stejný pro celý sloupec. Všechny záznamy mají v jednom sloupci hodnotu stejného typu. Databáze v MySQL nejsou výjimkou a platí pro ně totéž. Datový typ není závislý na typu tabulky. V dalších odstavcích si představíme nejdůležitější datové typy v MySQL

12.1.1 Celá čísla



V MySQL se standardně po všechna celá čísla používá znaménko. Pokud chceme do sloupce ukládat pouze kladná čísla, je nutné u definice sloupce zadat atribut UNSIGNED. V MySQL je k dispozici 6 datových typů pro celá čísla. Podle velikosti od nejmenšího po největší jsou to TINYINT, SMALLINT, MEDIUMINT, INT (nebo INTEGER) a BIGINT. Kromě toho existuje ještě typ SERIÁL, který slouží pro sloupec představující primární klíč. Typ TINYINT umožňuje uložit hodnoty -127 až +128 a zabírá jeden byte. Další datové typy mají vždy dvojnásobný rozsah než předchozí.

MySQL umožňuje sloupci, který je číselného typu, nastavit vlastnost AUTO_INCREMENT. Nastavení této vlastnosti je vhodné zejména u sloupců, které tvoří primární klíč (často se takový sloupec nazývá ID). Nastavením vlastnosti docílíme toho, že hodnoty ve sloupci se budou u každého nového záznamu zvyšovat o jedničku – tím pádem je každá hodnota jedinečná.

12.1.2 Desetinná čísla



Pro uložení desetinných čísel je možné použít čísla s pevnou nebo s plovoucí desetinnou čárkou. Rozdíl mezi nimi spočívá v jejich vnitřní interpretaci. Čísla s plovoucí desetinnou čárkou se v MySQL ukládají v binárním formátu, čísla s pevnou desetinnou čárkou se ukládají jako řetězce. U čísel s pevnou desetinnou čárkou jsou vyloučeny chyby způsobené zaokrouhlováním. Nevýhodou ale je, že jsou vyšší nároky na paměť.

MySQL obsahuje 3 datové typy pro desetinná čísla. Datové typy FLOAT a DOUBLE slouží pro uložení čísel s plovoucí desetinnou čárkou. Datový typ DECIMAL pak slouží k uložení čísel s pevnou desetinnou čárkou. Kromě toho existují typy REAL (synonymum pro DOUBLE), NUMERIC a DEC (oba představují synonymum pro DECIMAL). Pomocí parametrů můžeme u všech těchto typů nastavit počet cifer a počet desetinným míst.

12.1.3 Datum a čas



Pro reprezentaci kalendářního data a času se nejčastěji používá typ DATETIME. Pomocí tohoto typu můžeme ukládat libovolné časové údaje. Kromě toho existují typy DATE pro reprezentaci data (bez času) a TIME pro reprezentaci času (bez data). Dalšími typy jsou pak YEAR (letopočet) a TIMESTAMP. Obsah sloupce s typem TIMESTAMP se při každé změně v MySQL automaticky aktualizuje. Jeho použití je vhodné v případě, že chceme sledovat datum a čas poslední změny.

Datum v MySQL je uloženo ve formátu YYYY-mm-dd, tj. jako první je uložen rok, dále měsíc (vždy na dvě místa) a nakonec den (také na dvě místa). Oddělovačem je znak – (pomlčka). Datum 17.6.2010 bude tedy uloženo jako 2010-06-17, datum 6.11.2012 jako 2012-11-06.

12.1.4 Řetězce znaků



Pro ukládání řetězců znaků jsou v MySQL k dispozici datové typy CHAR, VARCHAR, TINYTEXT, TEXT, MEDIUMTEXT a LONGTEXT. U typů CHAR a VARCHAR můžeme nastavit maximální délku řetězce. U ostatních datových typů je maximální délka určena přímo datovým typem.

12.1.5 Binární data

Pro uložení binárních dat, např. obrázků, můžeme v MySQL použít 4 datové typy. Rozdíl mezi nimi je pouze ve velikosti. Pro binární data jsou určeny datové typy TINYBLOB, BLOB, MEDIUMBLOB a LONGBLOB.

12.2 Přístup k databázi MySQL prostřednictvím PHP

Pro přístup k databázi MySQL jsou v PHP k dispozici dvě různá rozhraní. Prvním způsobem je funkce `mysql`, která byla k dispozici už v dřívějších verzích PHP. Od PHP verze 5 je k dispozici také objektově orientované rozhraní `mysqli`. Rozhraní `mysqli` můžeme použít ale i procedurálním stylem. V této studijní opoře se zaměříme na rozhraní `mysqli`, ale zmíníme i funkce `mysql`. Rozhraní `mysqli` je novější a funkce `mysql` už začínají být zastaralé.

Obě rozhraní nejsou standardní součástí PHP. K jejich fungování je potřeba nainstalovat potřebné knihovny. Pokud potřebujeme zjistit, zda je v PHP nainstalována podpora MySQL, můžeme tak učinit pomocí funkce `phpinfo()`, kterou zavoláme v libovolném souboru.

12.2.1 Připojení k databázi a výběr databáze



Prvním krokem při práci s databází je připojení k databázovému serveru MySQL. Připojení pomocí `mysqli` je možné dvěma způsoby. Prvním způsobem je vytvoření instance objektu `mysqli`. Konstruktor může obsahovat až 6 volitelných parametrů. Jsou to adresa databázového serveru (pokud máme PHP i MySQL nainstalovány lokálně, je adresa `localhost`), jméno uživatele, heslo uživatele, název databáze, port a socket. Pokud je pokus o připojení k databázovému serveru úspěšný, je vytvořen objekt, která představuje připojení k MySQL serveru. Druhou možností připojení je pomocí funkce `mysqli_real_connect()`.

Ve starším rozhraní `mysql` jsou k dispozici funkce `mysql_connect()` a `mysql_pconnect()`. Všechny parametry těchto funkcí jsou volitelné. Pro obě jsou důležité první tři parametry. Prvním parametrem je adresa databázového serveru, druhým uživatelské jméno a třetím heslo. Obě funkce vracejí odkaz na připojení k MySQL, v případě neúspěchu pak vracejí `false`.



Po připojení k databázovému serveru je ještě potřeba vybrat databázi, se kterou budeme pracovat. Pokud databáze již existuje, pro její výběr použijeme funkci `mysql_select_db()`, jejímž parametrem je název databáze. Pokud databáze doposud neexistuje, můžeme ji vytvořit pomocí funkce `mysql_create_db()`. Parametrem je opět název databáze. Obě funkce mají druhý volitelný parametr a tím je odkaz na aktivní připojení k databázi. Návratovou hodnotou obou funkcí je logická hodnota, pokud je operace

úspěšná, funkce vracejí `true`, v případě neúspěchu vracejí `false`. Při použití rozhraní `mysqli` je název funkce pro výběr databáze `mysqli_select_db()`. Návratová hodnota je stejná jako u funkcí `mysql_select_db()` a `mysql_create_db()`. Parametry se liší, funkce má dva povinné parametry. První parametrem je odkaz na aktuální připojení k databázovému serveru, druhým parametrem je pak název databáze.

Následující výpis ukazuje připojení k databázovému serveru a výběr databáze pomocí obou rozhraní. Pro připojení je použit účet `root` (výchozí účet po nainstalování MySQL serveru), který je v aktuální konfiguraci bez hesla. Po připojení je jako aktuální zvolena databáze `moje_db`.

Výpis 12.1 Připojení k databázovému serveru.



```
<?php
  $pripojeni1 = mysql_pconnect("localhost", "root")
  or exit ("Není možné připojit databázový server");
  $db = mysql_select_db("moje_db")
  or exit ("Není možné vybrat databázi");

  $pripojeni2 = mysqli_init();
  if (!$pripojeni2) {
    exit("mysqli_init selhalo");
  } else {
    echo "mysql_init proběhlo v pořádku";
  }
  if (!mysqli_real_connect($pripojeni2, "localhost",
   "root")) {
    exit ("Není možné připojit databázový server");
  } else {
    echo "Připojení k databázovému serveru proběhlo
     úspěšně";
  }
  mysqli_select_db($pripojeni2, "moje_db")
  or exit ("Není možné vybrat databázi");
?>
```

12.2.2 Spouštění příkazů SQL



Po připojení k databázovému serveru a zvolení aktuální databáze již můžeme spouštět SQL příkazy. Může se jednat o jakékoliv příkazy z jazyka SQL. Můžeme použít příkazy z částí DDL (Data Definition Language), DML (Data Manipulation Language) i DQL (Data Query Language). Do části DDL patří příkazy pro vytvoření, úpravu nebo smazání databázových objektů, např.

tabulky, indexu nebo pohledu. V části DML jsou k dispozici příkazy pro manipulaci se záznamy tabulek. Můžeme vložit nový záznam (řádek) do tabulky, upravit jej nebo jej smazat. Pro část DQL je pak k dispozici pouze jeden příkaz a tím je příkaz SELECT, který slouží k vybrání záznamů z databáze, které splňují zadaná kritéria. Z hlediska spouštění dotazů pomocí PHP pak mohou nastat dvě situace. První situací je použití příkazu z části DDL nebo DML. Spuštění takového dotazu má jako efekt změnu databázového objektu nebo úpravu sady záznamů v tabulkách. Taková změna tedy nevrací žádný výsledek (i když z hlediska použití v PHP to není úplně přesně řečeno, viz. dále). Použití příkazu SELECT má naopak jako efekt načtení sady záznamů z tabulky nebo tabulek, které odpovídají zadaným kritériím. Tato data jsou pak vrácena ve formě jakési virtuální tabulky, která obsahuje sloupce definované v příkazu SELECT. Vrácená tabulka může být samozřejmě prázdná, pokud neexistuje záznam, který by odpovídal zadaným kritériím.



Pro spuštění libovolného SQL příkazu pomocí rozhraní mysqli slouží funkce `mysqli_query()`, která má dva parametry. Prvním parametrem je odkaz na připojení k databázovému serveru, druhým parametrem je dotaz, který chceme spustit. Dotaz se zadává ve formě textového řetězce. Návratová hodnota se liší podle toho, jaký SQL příkaz byl spuštěn. Jestliže byl spuštěn příkaz SELECT, funkce vrací objekt `mysqli_result`, který představuje již zmíněnou virtuální tabulku. Pro ostatní příkazy SQL funkce vrací `true` v případě úspěchu. V případě neúspěchu vrací funkce `false` a je jedno, jaký SQL příkaz byl spuštěn. Rozhraní `mysql` obsahuje funkci `mysql_query()`, která funguje velmi podobně.

Pokud v aktuální databázi spustíme některý z příkazů DDL nebo DML, stačí otestovat, zda funkce `mysqli_query()` vrátila hodnotu `true`. Následující výpis zobrazuje ukázku použití funkce `mysqli_query()`, v proměnné `$pripojeni` je uložen odkaz na aktuální připojení k databázovému serveru.

Výpis 12.2 Vytvoření tabulky a vložení záznamu pomocí funkce mysqli_query().



```
<?php
  $dotaz = "CREATE TABLE IF NOT EXISTS osoba (
   id_osoba int(11) NOT NULL AUTO_INCREMENT,
   jmeno varchar(50) NOT NULL,
   prijmeni varchar(50) NOT NULL,
   PRIMARY KEY (id_osoba)
   ) ENGINE=InnoDB DEFAULT CHARSET=utf8
   COLLATE=utf8_czech_ci;";
```

```
  $vysledek = mysqli_query($pripojeni, $dotaz);
  if ($vysledek) {
    echo "Tabulka byla úspěšně vytvořena.<br>";
  } else {
    echo "Chyba při vytváření tabulky.<br>";
  }

  $dotaz = "INSERT INTO osoba (jmeno, prijmeni)
   VALUES ('Rostislav', 'Miarka');";
  $vysledek = mysqli_query($pripojeni, $dotaz);
  if ($vysledek) {
    echo "Záznam byl úspěšně vložen.<br>";
  } else {
    echo "Chyba při vkládání záznamu.<br>";
  }
  $dotaz = "INSERT INTO osoba (jmeno, prijmeni)
   VALUES ('Jan', 'Novák');";
  $vysledek = mysqli_query($pripojeni, $dotaz);
  if ($vysledek) {
    echo "Záznam byl úspěšně vložen.<br>";
  } else {
    echo "Chyba při vkládání záznamu.<br>";
  }
  $dotaz = "INSERT INTO osoba (jmeno, prijmeni)
   VALUES ('Petra', 'Dvořáková');";
  $vysledek = mysqli_query($pripojeni, $dotaz);
  if ($vysledek) {
    echo "Záznam byl úspěšně vložen.<br>";
  } else {
    echo "Chyba při vkládání záznamu.<br>";
  }
  $dotaz = "INSERT INTO osoba (jmeno, prijmeni)
   VALUES ('David', 'Král');";
  $vysledek = mysqli_query($pripojeni, $dotaz);
  if ($vysledek) {
    echo "Záznam byl úspěšně vložen.<br>";
  } else {
    echo "Chyba při vkládání záznamu.<br>";
  }
  $posledniId = mysqli_insert_id($pripojeni);
  echo "Poslední ID: " . $posledniId;
?>
```

V uvedeném výpisu je použit postup, kdy je prováděný SQL příkaz uložen do samostatné proměnné a při volání funkce `mysqli_query` je tato proměnná předána jako parametr. První příkaz vytvoří tabulku osoba, která má 3 sloupce.

Sloupec `id_osoba` tvoří primární klíč a má nastavenu vlastnost AUTO_INCREMENT (při každém vložení záznamu je vygenerována nová hodnota v tomto sloupci o jedničku vyšší než předchozí hodnota, první záznam má ve sloupci hodnotu 1). Všimněte si, že název tabulky nebo názvy sloupců nemusí být uvedeny v uvozovkách ani v apostrofech. Pokud bychom chtěli použít apostrof, je potřeba použít znak ` (obrácený apostrof), jehož kód v ASCII tabulce je 96. S obyčejným apostrofem nebo s klasickými uvozovkami nebude příkaz fungovat. Při vkládání záznamů do tabulky stačí vložit hodnoty do sloupců `jmeno` a `prijmeni`, do sloupce `id_osoba` se hodnota vygeneruje automaticky.

Pokud používáme u primárního klíče vlastnost AUTO_INCREMENT a potřebujeme zjistit hodnotu ID u posledního vloženého záznamu, můžeme použít funkci `mysqli_insert_id()`, jejímž parametrem je aktuální připojení k databázovému serveru. Ukázka použití této funkce je na posledních dvou řádcích výpisu.

12.2.3 Zpracování výsledků příkazu SELECT



Základem práce s každou databází je dotazování. Těžko bychom používali databázi pouze k tomu, abychom do ní uložili data a nijak s nimi nepracovali. Potřebujeme načítat data z tabulek a dále s nimi pracovat. K dotazování slouží příkaz SELECT. Jeho základní syntaxe je následující:

SELECT seznam_sloupců

FROM seznam_tabulek

WHERE podmínka

ORDER BY sloupce

Za slovem SELECT následuje seznam sloupců databázových tabulek, které chceme v dotazu zobrazit. Dále následuje slovo FROM a za ním seznam tabulek, ze kterých načítáme data. Další částí je stanovení podmínek, tato část následuje za slovem WHERE. Podmínek může být více, může se jednat o podmínky pro spojení tabulek nebo pro omezení výsledků. Poslední částí je část následující za slovy ORDER BY, v této části můžeme definovat řazení výsledků.

Pokud jako parametr funkce `mysqli_query()` použijeme dotaz definovaný pomocí příkazu SELECT, vrací tato funkce objekt `mysqli_result`. Tento

objekt představuje virtuální tabulku, kterou tvoří sloupce definované v dotazu a která obsahuje záznamy splňující zadaná kritéria. S výsledkem dotazu můžeme dále pracovat. Jsou v zásadě dvě možnosti. Buď můžeme pracovat přímo s objektem `mysqli_result` a nebo s jeho částmi (řádky nebo sloupce). V případě zpracování po částech jen nejtypičtější zpracování po řádcích. Pro zpracování po řádcích je možné použít čtyři různé funkce. Všechny funkce mají jako parametr objekt, který reprezentuje výsledek dotazu. Funkce vracejí jeden řádek výsledku ve formě pole nebo objektu. Kromě toho se provede posun ukazatele v rámci výsledku na další záznam. Pokud už bylo dosaženo posledního záznamu, funkce vrací hodnotu `false`. K procházení výsledků můžeme použít následující funkce:



- `mysqli_fetch_row()` – vrací záznam (řádek) výsledku ve formě jednoduchého pole. Jednotlivé sloupce výsledku jsou indexovány číslem, první sloupec má index `0`.

- `mysqli_fetch_assoc()` – vrací záznam (řádek) výsledku ve formě asociativního polo. K jednotlivým sloupcům výsledku můžeme přistupovat pomocí indexu tvořeného řetězcem – indexem je název sloupce tabulky.

- `mysqli_fetch_array()` – vrací záznam (řádek) výsledku ve formě jednoduchého a asociativního polo. K jednotlivým sloupcům výsledku můžeme přistupovat buď pomocí číselného indexu (první sloupec má opět index `0`) nebo pomocí indexu tvořeného řetězcem – indexem je název sloupce tabulky.

- `mysqli_fetch_object()` – vrací záznam (řádek) výsledku ve formě objektu. Jednotlivé sloupce výsledku pak tvoří atributy (členské proměnné) objektu. K hodnotám ve sloupcích pak můžeme přistupovat pomocí názvu sloupců. Pokud je již dosaženo konce výsledku, vrací funkce hodnotu `null` (na rozdíl od předchozích třech funkcí).

Při zpracování výsledků dotazu je někdy užitečné znát počet záznamů ve výsledku. Ke zjištění počtu řádku v objektu `mysqli_result` slouží funkce `mysqli_num_rows()`, jejímž parametrem je instance objektu `mysqli_result`. Funkce vrací číslo – počet řádků ve výsledku.

Kromě uvedených funkcí můžeme u instance objektu `mysqli_result` zjišťovat počet sloupců, zjišťovat metadata o sloupcích, testovat, zda sloupec

existuje apod. Nejčastěji ale použijeme výše zmíněné funkce, protože při dotazování bychom měli znát strukturu databáze, takže víme, jaké sloupce jsou v tabulkách. Bez této informace bychom těžko vytvářeli složitější příkazy SELECT.

Následující výpis ukazuje načtení obsahu tabulky osoba. Vytvoření této tabulky a vložení 4 záznamů je uvedeno v předchozím výpisu. Ve výpisu je opět použita proměnná `$pripojeni`, která obsahuje odkaz na aktuální připojení k databázovému serveru. Obrázek pod výpisem pak zobrazuje výsledek v prohlížeči.

Výpis 12.3 Zpracování výsledku dotazu SELECT.

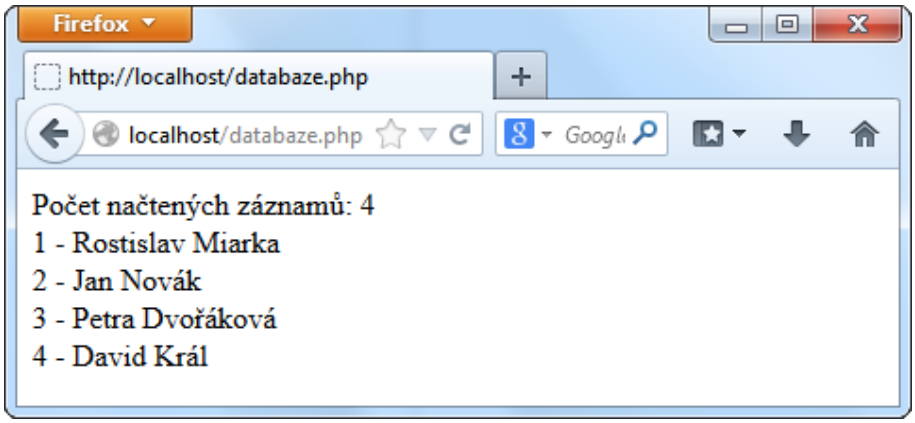


```
<?php
  $dotaz = "SELECT * FROM osoba";
  $vysledek = mysqli_query($pripojeni, $dotaz);
  $pocet = mysqli_num_rows($vysledek);
  echo "Počet načtených záznamů: " . $pocet . "<br>";
  // klasicke pole
  $radek = mysqli_fetch_row($vysledek);
  echo $radek[0] . " - " . $radek[1] . " " .
   $radek[2] . "<br>";

  // asociativni pole
  $radek = mysqli_fetch_assoc($vysledek);
  echo $radek["id_osoba"] . " - " . $radek["jmeno"] .
   " " . $radek["prijmeni"] . "<br>";

  // klasicke a asociativni pole
  $radek = mysqli_fetch_array($vysledek);
  echo $radek["id_osoba"] . " - " . $radek[1] . " " .
   $radek["prijmeni"] . "<br>";

  // objekt
  $radek = mysqli_fetch_object($vysledek);
  echo $radek->id_osoba . " - " . $radek->jmeno . " "
   . $radek->prijmeni . "<br>";
?>
```



Obrázek 12.1 Zpracování výsledku dotazu SELECT.

12.2.4 Uvolnění výsledku a uzavření připojení k databázi.



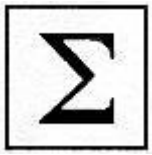
Po ukončení práce s jakýmkoliv zdrojem je potřeba uzavřít veškerá existující připojení. Pokud pracujeme se souborem, po skončení práce musíme soubor uzavřít. Totéž platí pro práci s databází. PHP umožňuje uvolnit (zrušit) výsledek, který byl vrácen po provedení příkazu SELECT. K tomuto účelu je určena funkce `mysqli_free_result()`, která má jako parametr instanci objektu `mysqli_result`. Funkce nic nevrací, tj. vrací datový typ `void`.

Po uvolnění výsledku je připojení k databázovému serveru stále aktivní. Ke korektnímu ukončení práce je potřeba připojení uzavřít. K uzavření připojení slouží funkce `mysqli_close()`, jejímž parametrem je odkaz na aktivní připojení k databázi. Funkce vrací logickou hodnotu. Pokud vše proběhne bez problémů, vrátí hodnotu `true`, v případě neúspěchu pak hodnotu `false`.



Kontrolní otázky:

1. Pomocí jakých rozhraní je možné pracovat v PHP s databází MySQL?
2. Jak provedete připojení k databázi a výběr databáze?
3. Jaké funkce je možné použít pro zpracování výsledků dotazu SELECT?



Shrnutí obsahu kapitoly

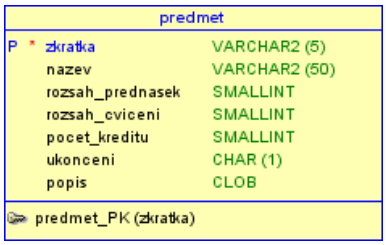
V této kapitole jste se seznámili se základními principy práce s databází MySQL pomocí jazyka PHP. K práci s databází je možné použít dvě rozhraní `mysql` a `mysqli`. Rozhraní `mysqli` je novější, proto je vhodnější. Základní funkcí pro spouštění příkazů SQL je `mysqli_query()`. Pomocí této funkce můžeme spouštět libovolné SQL příkazy. Ke zpracování výsledků vrácených

po spuštění příkazu SELECT slouží objekt `mysqli_result`. Procházení záznamů ve výsledku je možné pomocí čtyř různých funkcí. Po ukončení práce s databází bychom měli uvolnit načtené výsledky a uzavřít připojení k databázi.

Korespondenční úkol č. 2



Na databázovém serveru si vytvořte tabulku předmětů, která bude mít strukturu jako na následujícím obrázku (pro jednotlivé sloupce použijte vhodné datové typy MySQL):



Dále vytvořte PHP skript, který bude obsahovat formulář pro vložení záznamu do tabulky `predmet`. Ve formuláři použijte vhodné formulářové prvky. Pod formulářem bude ve formě tabulky vypsán obsah tabulky `predmet`. Pokud tabulka neobsahuje žádný záznam, bude o tom zobrazena zpráva.

Formulář se bude odesílat na stejný PHP skript. Po odeslání formuláře se provede kontrola zadaných dat (pokud bude nutná) a uloží se nový záznam do tabulky.